

Machine Learning: Optimization, Bias-Variance Decomposition, and Regularization

Quoc Kien

1. Optimization via Gradient Descent Variants

To minimize our loss function $L(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$, we use iterative optimization. The general parameter update rule for a single feature index $j \in \{1, 2, \dots, p\}$ at iteration t is:

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \alpha \frac{\partial L(\beta^{(t)})}{\partial \beta_j}$$

Where α is the learning rate.

Gradient Derivation

Taking the partial derivative of the loss function for a single parameter component β_j :

$$\frac{\partial L(\beta)}{\partial \beta_j} = \frac{1}{2n} \sum_{i=1}^n 2(y_i - \beta^T \mathbf{x}_i) \cdot \frac{\partial (y_i - \beta^T \mathbf{x}_i)}{\partial \beta_j}$$

$$\frac{\partial L(\beta)}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)(-x_{i,j})$$

$$\frac{\partial L(\beta)}{\partial \beta_j} = -\frac{1}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)x_{i,j}$$

Substituting this back into the gradient descent update equation flips the negative sign to a positive step:

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \frac{\alpha}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)x_{i,j}$$

Comparison of Optimization Regimes

Depending on how many samples we look at before updating our weights, we can group gradient descent into three variants:

Variant	Data Subset per Update	Mathematical Update Step	Operational Trade-off
Batch Gradient Descent (BCD)	Entire dataset (n samples)	$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i) x_{i,j}$	Guaranteed smooth convergence to global minimum, but highly computationally expensive and slow for massive datasets.
Stochastic Gradient Descent (SGD)	A single random sample $i \in \{1, \dots, n\}$	$\beta_j \leftarrow \beta_j + \alpha (y_i - \beta^T \mathbf{x}_i) x_{i,j}$	Fast execution and low memory usage. However, updates exhibit high variance, introducing significant noise that causes the loss curve to oscillate heavily.
Mini-batch Stochastic Gradient Descent	A random subset $\mathcal{B} \subset \{1, \dots, n\}$ of size m	$\beta_j \leftarrow \beta_j + \frac{\alpha}{m} \sum_{i \in \mathcal{B}} (y_i - \beta^T \mathbf{x}_i) x_{i,j}$	Best of both worlds. Strikes an ideal balance between computational velocity, vectorization efficiency, and stable empirical convergence.

2. Generalization: Underfitting vs. Overfitting

When evaluating a model parameter vector β on an unseen test dataset $\mathcal{D}_{\text{test}}$ sampled from the true underlying distribution $\mathcal{D}$, we analyze the average test loss:

$$L_{\text{test}}(\beta) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - \hat{f}_{\beta}(\mathbf{x}))^2 \right]$$

The Two Failure Modes

1. Underfitting (High Bias):

- **Symptom:** Both training error and test error are exceptionally large.
- **Cause:** The chosen model architecture lacks structural capacity and is too simple to capture the underlying pattern of the data.
- **Weights:** Parameter magnitudes $\|\beta\|$ are often very small or suppressed.

2. Overfitting (High Variance):

- **Symptom:** Training error is remarkably low, but test error skyrockets.
 - **Cause:** The model is overly complex (over-parameterized, where number of parameters $p \gg n$). It memorizes the random training noise instead of discovering generalizable patterns.
 - **Weights:** Parameter values $\|\beta\|$ fluctuate erratically with extreme magnitudes.
-

3. Mathematical Proof of Bias-Variance Decomposition

To understand why a model fails to generalize, we decompose the Mean Squared Error (MSE) at a specific unseen test point $\mathbf{x}$ into three distinct mathematical components.

Setup and Assumptions

- True generative function: $y = f(\mathbf{x}) + \epsilon$
- Noise properties: $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2$ (Intrinsic Irreducible Error)
- Estimated model trained on a random dataset $\mathcal{D}$: $\hat{f}(\mathbf{x}; \mathcal{D})$ (written as $\hat{f}$ for brevity).

Note that across random dataset realizations, the true function value $f(\mathbf{x})$ is a fixed constant, meaning $\mathbb{E}_{\mathcal{D}}[f(\mathbf{x})] = f(\mathbf{x})$.

The Step-by-Step Derivation

We start by expanding the expected prediction error:

$$\mathbb{E}_{\mathcal{D}, \epsilon} [(y - \hat{f})^2] = \mathbb{E}_{\mathcal{D}, \epsilon} [(f(\mathbf{x}) + \epsilon - \hat{f})^2]$$

$$\mathbb{E}_{\mathcal{D}, \epsilon} [(f(\mathbf{x}) - \hat{f})^2 + 2\epsilon(f(\mathbf{x}) - \hat{f}) + \epsilon^2]$$

Since the noise ϵ is independent of the training data $\mathcal{D}$, the expectation of the cross-product simplifies to zero:

$$\mathbb{E}_{\mathcal{D}, \epsilon}[2\epsilon(f(\mathbf{x}) - \hat{f})] = 2\mathbb{E}[\epsilon] \cdot \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}) - \hat{f}] = 0.$$

Additionally, by definition, $\mathbb{E}[\epsilon^2] = \text{Var}(\epsilon) = \sigma^2$. This leaves:

$$\mathbb{E}_{\mathcal{D}} \left[(f(\mathbf{x}) - \hat{f})^2 \right] + \sigma^2 \quad (\text{Equation 1})$$

Now, let us apply the algebraic trick of adding and subtracting the expected model prediction $\mathbb{E}_{\mathcal{D}}[\hat{f}]$ inside the remaining squared expression:

$$\mathbb{E}_{\mathcal{D}} \left[\left(f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}] + \mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f} \right)^2 \right]$$

Let $A = f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}]$ (which is a constant with respect to $\mathcal{D}$) and $B = \mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f}$. Expanding $(A + B)^2$:

$$\mathbb{E}_{\mathcal{D}} [A^2 + 2AB + B^2] = \mathbb{E}_{\mathcal{D}}[A^2] + 2\mathbb{E}_{\mathcal{D}}[AB] + \mathbb{E}_{\mathcal{D}}[B^2]$$

Let's evaluate each component individually: * $\mathbb{E}_{\mathcal{D}}[A^2] = (f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}])^2 = \text{Bias}^2(\hat{f})$ * $2\mathbb{E}_{\mathcal{D}}[AB] = 2(f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}]) \cdot \mathbb{E}_{\mathcal{D}}[\mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f}]$.

Since $\mathbb{E}_{\mathcal{D}}[\mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f}] = \mathbb{E}_{\mathcal{D}}[\hat{f}] - \mathbb{E}_{\mathcal{D}}[\hat{f}] = 0$, this entire cross-term vanishes. * $\mathbb{E}_{\mathcal{D}}[B^2] = \mathbb{E}_{\mathcal{D}}[(\mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f})^2] = \text{Var}(\hat{f})$

Combining all evaluated terms yields the definitive structural equation:

$$\text{Total Expected Error} = \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}) + \sigma^2$$

4. Regularization: Managing Overfitting

To prevent complex models from fitting noise, we append an explicit structural penalty $R(\beta)$ to our objective cost function. This forces the optimization to favor smoother patterns with smaller parameter weights:

$$L_{\text{reg}}(\beta) = L(\beta) + \lambda R(\beta)$$

Where $\lambda \geq 0$ is a hyperparameter that controls the regularization strength.

Explicit Regularization Paradigms

1. Ridge Regularization (L_2 Penalty)

- **Penalty term:** $R(\beta) = \|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$
- **Behavior:** Penalizes the square of parameter magnitudes. It shrinks all coefficients towards zero uniformly, mitigating high variance while maintaining all features.

2. Lasso Regularization (L_1 Penalty)

- **Penalty term:** $R(\beta) = \|\beta\|_1 = \sum_{j=1}^p |\beta_j|$
- **Behavior:** Penalizes absolute magnitudes. Due to the geometric nature of absolute values, it drives less informative weights exactly to zero, performing intrinsic feature selection.

Implicit Regularization

Over-parameterized regimes ($p \gg n$) often exhibit implicit regularization. For instance, initialization choices or optimization path bounds in certain architectures naturally encourage the algorithm to converge toward smooth, low-norm solutions without an explicit penalty term.

5. Hyperparameter Selection: Cross-Validation

To find the optimal regularization balance ($\lambda = \{\lambda_1, \lambda_2, \dots, \lambda_k\}$), we partition our training data to simulate unseen environments without touching our final evaluation test set.

K -Fold Cross-Validation Protocol

1. Split the primary training data partition into K equal, distinct folds.
2. For each unique hyperparameter candidate λ_j :
 - Iterate K times, treating a different fold as the validation set while training on the remaining $K - 1$ folds.
 - Record the average validation score across all K iterations.
3. Select the λ candidate that achieves the lowest average validation loss, then re-train the global model on the entire training set using that optimal hyperparameter value.